Comprehensive Documentation: AI Agents with Google Maps

Traditional Function Tools vs Model Context Protocol (MCP)

Table of Contents

Project Overview

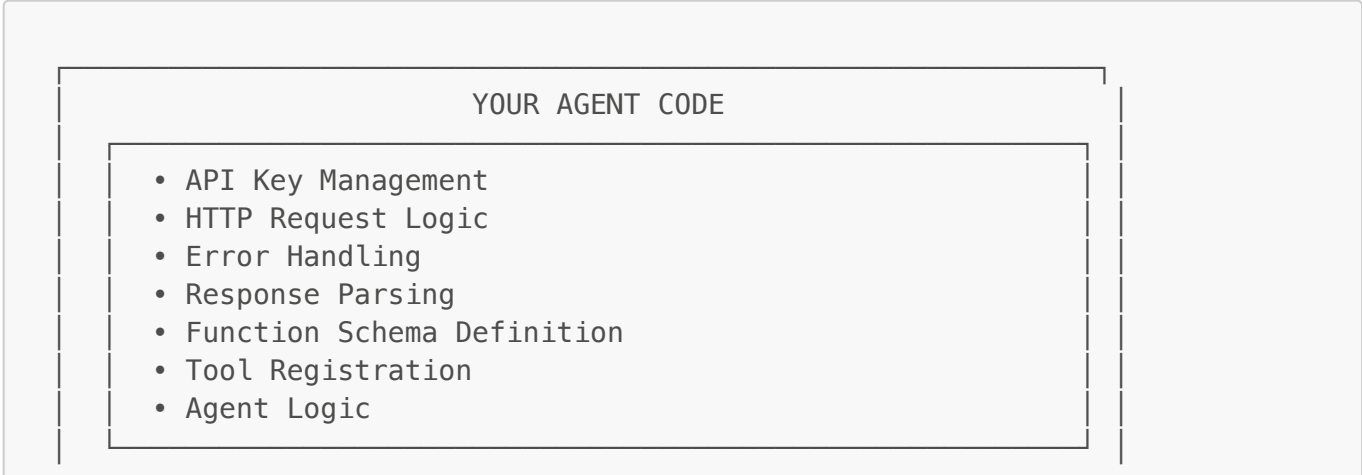
This project demonstrates two different approaches to building AI agents that interact with Google Maps:

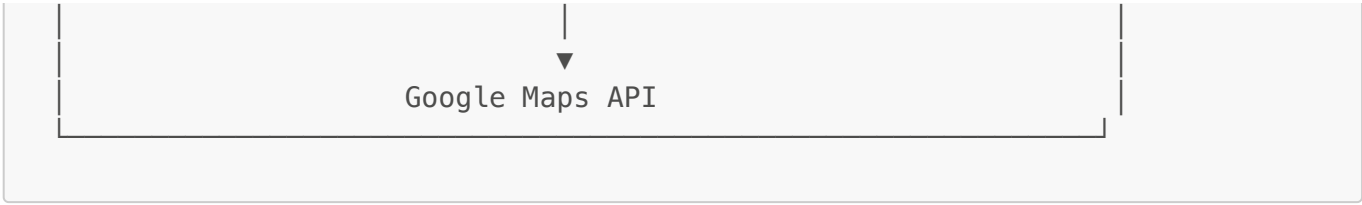
Agent	Approach	Tools Used
`Maps_Agent`	Traditional Function Tool Calling	Custom functions: `get_directions()`, `search_places_on_route()`
`MCP_Maps_Agent`	Model Context Protocol (MCP)	Google Maps MCP Server with 3 built-in tools

Both agents use **Google's Agent Development Kit (ADK)** and the **Gemini 2.0 Flash** model.

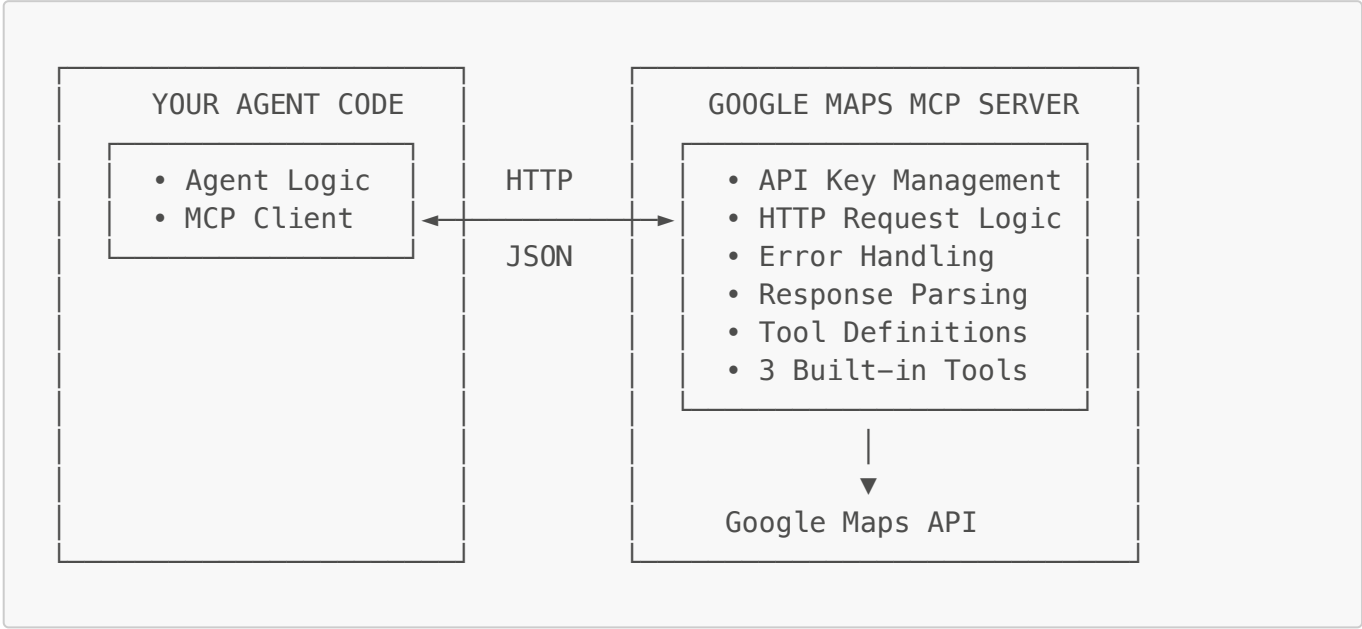
Architecture Comparison

Traditional Approach (Tight Coupling)





MCP Approach (Loose Coupling)



Traditional Approach: Maps_Agent

File: `Maps_Agent/agent.py`

Let's break down each component:

Step 1: Imports and Environment Setup

```
import os
import requests
from dotenv import load_dotenv
from google.adk.agents import LlmAgent
from google.adk.tools.function_tool import FunctionTool

load_dotenv()
MAPS_API_KEY = os.getenv("MAPS_API_KEY")
```

What's happening:

- `requests`: HTTP library to make API calls to Google Maps
- `load_dotenv()`: Loads environment variables from `.env` file
- `LlmAgent`: The core agent class from Google ADK
- `FunctionTool`: Wrapper to convert Python functions into agent-callable tools
- `MAPS_API_KEY`: Your Google Maps API key stored in environment

⚠ **Tight Coupling Point #1:** You must manage the API key in your agent code.

Step 2: The Tool Function (API Integration)

```
def get_directions(origin: str, destination: str, mode: str = "driving") -
> dict:
    """Get route summary using Google Directions API."""
    url = "https://maps.googleapis.com/maps/api/directions/json"
    r = requests.get(
        url,
        params={"origin": origin, "destination": destination, "mode":
mode, "key": MAPS_API_KEY},
        timeout=20,
    )
    r.raise_for_status()
    data = r.json()

    if data.get("status") != "OK" or not data.get("routes"):
        return {"status": data.get("status"), "error":
data.get("error_message", "No route found")}

    leg = data["routes"][0]["legs"][0]
    return {
        "origin": leg["start_address"],
        "destination": leg["end_address"],
        "distance": leg["distance"]["text"],
        "duration": leg["duration"]["text"],
        "google_maps_link": f"https://www.google.com/maps/dir/?
api=1&origin={origin}&destination={destination}&travelmode={mode}",
    }
```

What's happening:

1. **Function Signature**: Parameters become the tool's input schema that LLM understands
    - `origin: str` → Required input
    - `destination: str` → Required input
    - `mode: str = "driving"` → Optional with default
2. **Docstring**: `"""Get route summary..."""` becomes the tool's description for the LLM
3. **HTTP Request**: Direct API call to Google Maps Directions API
4. **Response Parsing**: Extract and format relevant data from API response

⚠ **Tight Coupling Points:**

- **#2:** HTTP request logic is in your code
- **#3:** Error handling is your responsibility
- **#4:** Response parsing requires understanding API structure

- **#5:** URL construction and parameter mapping is hardcoded

---

Step 3: Tool Registration

```
directions_tool = FunctionTool(get_directions)
```

What's happening:

- `FunctionTool` wraps your Python function
- It automatically extracts:
  - Function name → Tool name
  - Type hints → Parameter schema
  - Docstring → Tool description
- The LLM uses this schema to understand when and how to call the tool

---

Step 4: Agent Definition

```
root_agent = LlmAgent(
    model="gemini-2.0-flash",
    name="Maps_Agent",
    instruction=(
        "You are a helpful Maps assistant.\n"
        "Use appropriate tools to answer user's query:\n"
        "1) Get directions between two locations\n"
        "Always return a short answer + the Google Maps link when
directions are requested."
    ),
    tools=[directions_tool]
)
```

What's happening:

- `model`: The LLM that powers the agent
- `name`: Identifier for the agent
- `instruction`: System prompt that guides agent behavior
- `tools`: List of tools the agent can use

At this point, we have a working agent with **one tool**. But what happens when we need to add more capabilities?

🚨 The Problem: Adding More Tools

Now imagine a user asks: *"Find me gas stations on my way from Chennai to Bangalore"*

Our current agent can't help—it only knows how to get directions. We need to add a **Places Search** capability.

What's Required to Add a Second Tool?

To add this one new capability, we must:

1. ✍️ **Research the API** - Understand Google Places Nearby Search API
2. ✍️ **Write HTTP logic** - Construct the request with correct parameters
3. ✍️ **Handle errors** - What if the API fails?
4. ✍️ **Parse responses** - Extract relevant data from nested JSON
5. ✍️ **Define the function** - With proper type hints and docstring
6. ✍️ **Wrap with FunctionTool** - Register it as a tool
7. ✍️ **Update the agent** - Add to tools list and update instructions

That's **7 steps** just to add ONE new tool!

Step 5: Adding the Places Search Tool

Here's the new function we need to write:

```
# Google Maps - Places Nearby Search API
def search_places_on_route(origin: str, destination: str, place_type: str)
-> dict:
    """Search for places along the route between two locations.

    Args:
        origin: Starting location
        destination: End location
        place_type: What to search for (e.g., 'gas station', 'restaurant',
'hotel', 'atm', 'hospital')
    """
    # Get route to find midpoint
    dir_response = requests.get(
        "https://maps.googleapis.com/maps/api/directions/json",
        params={"origin": origin, "destination": destination, "key":
MAPS_API_KEY},
        timeout=20,
    )
    dir_data = dir_response.json()

    if dir_data.get("status") != "OK":
        return {"error": "Could not find route"}

    # Get midpoint from route
    steps = dir_data["routes"][0]["legs"][0]["steps"]
    mid = steps[len(steps) // 2]["end_location"]

    # Search for places near midpoint
    places_response = requests.get(
        "https://maps.googleapis.com/maps/api/place/nearbysearch/json",
```

```
            params={
                "location": f"{mid['lat']},{mid['lng']}",
                "radius": 5000,
                "keyword": place_type,
                "key": MAPS_API_KEY,
            },
            timeout=20,
        )
        places_data = places_response.json()

        if not places_data.get("results"):
            return {"error": f"No {place_type} found along the route"}

        # Return top 5 places
        places = []
        for p in places_data["results"][:5]:
            places.append({
                "name": p.get("name"),
                "rating": p.get("rating", "N/A"),
                "address": p.get("vicinity"),
                "maps_link": f"https://www.google.com/maps/place/?q=place_id:
{p.get('place_id')}",
            })

        return {"query": place_type, "places": places}
```

Notice the complexity:

- We need to call **TWO different APIs** (Directions + Places)
- We must understand the response structure of BOTH APIs
- We're parsing nested JSON: `dir_data["routes"][0]["legs"][0]["steps"]`
- We handle errors for both API calls
- We construct a different URL with different parameters

⚠ New Tight Coupling Points:

- **#7:** Second API endpoint hardcoded
- **#8:** Second set of parameters to manage
- **#9:** More response parsing logic
- **#10:** Interdependency between APIs (need directions to find midpoint for places)

Step 6: Registering the Second Tool

```
directions_tool = FunctionTool(get_directions)
places_tool = FunctionTool(search_places_on_route)
```

Step 7: Updating the Agent

```
root_agent = LlmAgent(
    model="gemini-2.0-flash",
    name="Maps_Agent",
    instruction=(
        "You are a helpful Maps assistant.\n"
        "Use appropriate tools to answer user's query:\n"
        "1) Get directions between two locations\n"
        "2) Search for places (gas stations, restaurants, hotels, etc.)
along a route\n"
        "Always return a short answer + the Google Maps link."
    ),
    tools=[directions_tool, places_tool]
)
```

📊 The Growing Complexity Problem

Metric	1 Tool	2 Tools	5 Tools	10 Tools
Lines of code	~50	~100	~250+	~500+
APIs to understand	1	2	5	10
Error handling points	2	4	10+	20+
Response structures to parse	1	2	5	10
Maintenance burden	Low	Medium	High	Very High

The pattern is clear: With traditional function tools, complexity grows **linearly** with each new tool. Your agent code becomes:

- Harder to maintain
- More prone to bugs
- Tightly coupled to external API changes
- Difficult for teams to work on

This is exactly the problem MCP solves.

Modern Approach: MCP_Maps_Agent

File: `MCP_Maps_Agent/agent.py`

Step 1: Imports and Environment Setup

```
import os
from dotenv import load_dotenv

from google.adk.agents import LlmAgent
```

```
from google.adk.tools.mcp_tool.mcp_toolset import MCPToolset
from google.adk.tools.mcp_tool.mcp_session_manager import
StreamableHTTPConnectionParams

load_dotenv()
MAPS_API_KEY = os.getenv("MAPS_API_KEY")
```

What's happening:

- `MCPToolset`: A toolset that connects to an MCP server
- `StreamableHTTPConnectionParams`: Configuration for HTTP-based MCP connection
- **Notice:** No `requests` library needed! No HTTP logic in your code!

Step 2: MCP Server Connection

```
MAPS_MCP_URL = "https://mapstools.googleapis.com/mcp"  # Google-hosted
Maps MCP endpoint

maps_toolset = MCPToolset(
    connection_params=StreamableHTTPConnectionParams(
        url=MAPS_MCP_URL,
        headers={
            "X-Goog-Api-Key": MAPS_API_KEY,
        },
    )
)
```

What's happening:

1. **MCP Server URL**: Points to Google's hosted MCP server for Maps
2. **Authentication**: API key passed via headers (MCP server handles actual API calls)
3. **MCPToolset**: Automatically discovers and registers all tools from the server

✅ **Decoupling Benefits:**

- No HTTP request logic
- No response parsing
- No error handling for API calls
- Tools are discovered dynamically

Step 3: Agent Definition

```
root_agent = LlmAgent(
    model="gemini-2.0-flash",
    name="maps_mcp_agent",
    instruction=(
```

```
        "You are a helpful Maps assistant.\n"
        "Use the MCP-provided Maps tools to find places and get
directions.\n"
        "When directions are requested, include a Google Maps link in the
final answer."
    ),
    tools=[maps_toolset]
)
```

What's happening:

- Same structure as traditional approach
- `tools=[maps_toolset]` → One toolset provides multiple tools!
- Agent automatically knows about all tools from the MCP server

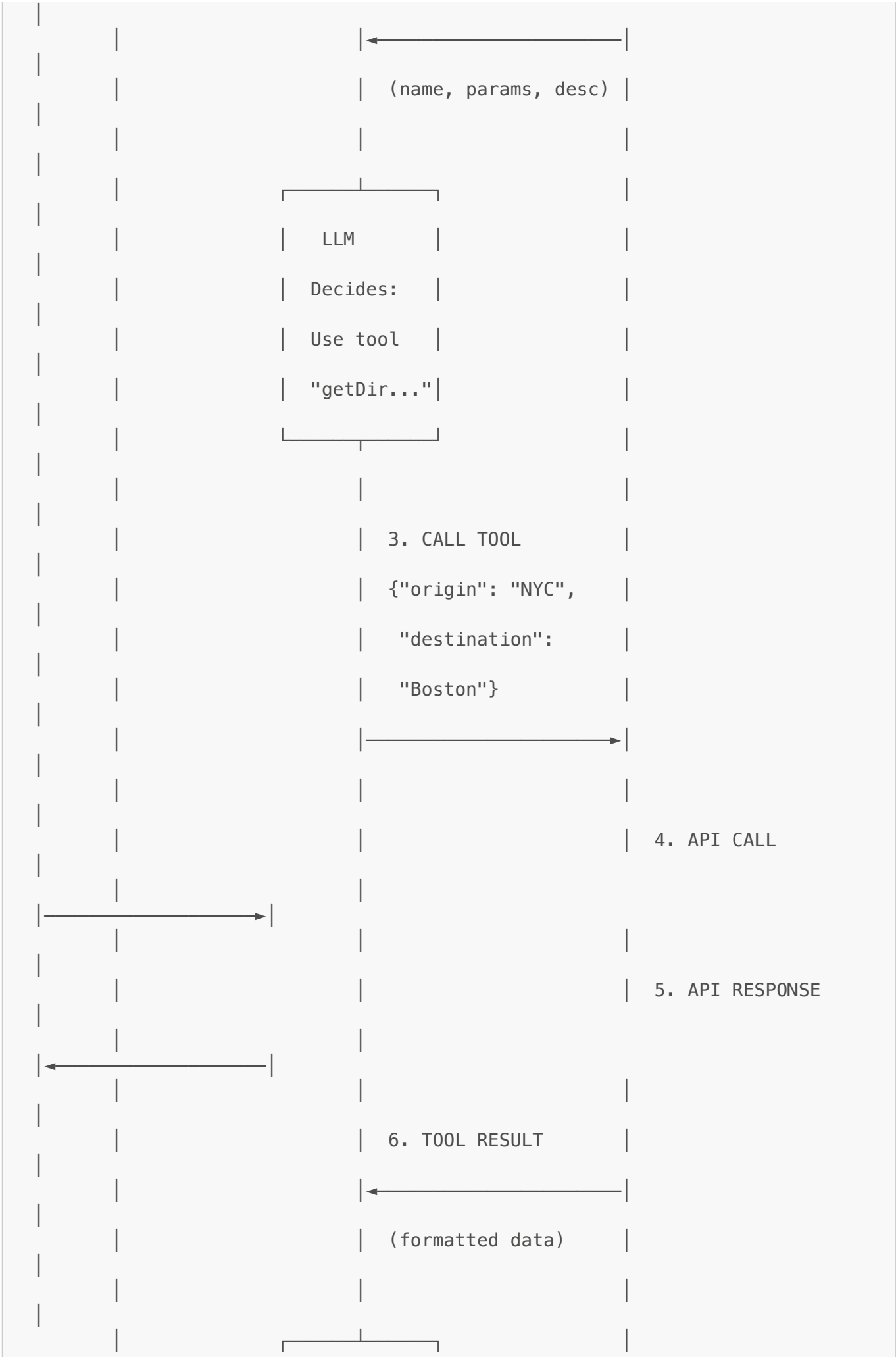
---

MCP Deep Dive: How It Works

What is Model Context Protocol (MCP)?

MCP is an **open standard** that enables AI applications to connect to external data sources and tools through a **unified protocol**. Think of it as a "USB-C for AI" – one standard connector for all tools.

The MCP Communication Flow


(name, params, desc)
LLM
Decides:
Use tool
"getDir..."
3. CALL TOOL
{"origin": "NYC",
"destination":
"Boston"}
4. API CALL
5. API RESPONSE
6. TOOL RESULT
(formatted data)

```
│
        │              │   LLM       │                │
│
        │              │ Generates   │                │
│
        │              │ Response    │                │
│
        │              └─────┬───────┘                │
│
        │                    │                        │
│
        │  "The route from   │                        │
│
        │   NYC to Boston..." │                       │
│
        │◄────────────────────┘                       │
│
```

Key MCP Concepts

1. Tool Discovery (How LLM Knows What Tools Exist)

When `MCPToolset` connects to the MCP server, it sends a `tools/list` request:

```
// Request to MCP Server
{
  "jsonrpc": "2.0",
  "method": "tools/list",
  "id": 1
}

// Response from MCP Server
{
  "jsonrpc": "2.0",
  "result": {
    "tools": [
      {
        "name": "maps_directions",
        "description": "Get directions between two locations",
        "inputSchema": {
          "type": "object",
          "properties": {
            "origin": {"type": "string", "description": "Starting
location"},
            "destination": {"type": "string", "description": "End
location"},
            "mode": {"type": "string", "enum": ["driving", "walking",
"bicycling", "transit"]}
          },
          "required": ["origin", "destination"]
        }
```

```
      },
      {
        "name": "maps_search_places",
        "description": "Search for places near a location",
        "inputSchema": {...}
      },
      {
        "name": "maps_geocode",
        "description": "Convert address to coordinates",
        "inputSchema": {...}
      }
    ]
  }
}
```

The LLM receives these tool definitions and understands:

- **What tools exist** (names)
- **What each tool does** (descriptions)
- **What parameters each tool needs** (inputSchema)

2. Tool Invocation (How Agent Calls MCP Tools)

When the LLM decides to use a tool, the agent sends a `tools/call` request:

```
// Request to MCP Server
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "maps_directions",
    "arguments": {
      "origin": "New York City",
      "destination": "Boston",
      "mode": "driving"
    }
  },
  "id": 2
}

// Response from MCP Server
{
  "jsonrpc": "2.0",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "{\"distance\": \"215 mi\", \"duration\": \"3h 45m\",
...}"
      }
    ]
```

```
    }
}
```

3. Protocol Format (JSON-RPC 2.0)

MCP uses **JSON-RPC 2.0** for communication:

- `jsonrpc`: Protocol version
- `method`: The action to perform
- `params`: Parameters for the method
- `id`: Request identifier for matching responses
- `result`: The response data

Google Maps MCP Server - Built-in Tools

The Google Maps MCP Server at `https://mapstools.googleapis.com/mcp` provides these tools:

Tool	Description	Key Parameters
`directions`	Get driving/walking/transit directions	origin, destination, mode
`search_places`	Find places (restaurants, hotels, etc.)	query, location, radius
`geocode`	Convert address to lat/lng coordinates	address

Key Differences and Benefits

Code Comparison

Aspect	Traditional (2 Tools)	MCP Approach (3+ Tools)
Lines of Code	~100 lines	~30 lines
HTTP Logic	2 API endpoints in your code	Handled by MCP server
Error Handling	4+ error points to manage	MCP server responsibility
Response Parsing	Parse 2 different API structures	Pre-formatted by MCP
Adding New Tools	~50 lines per tool	Already included!
API Updates	Update your code for each API	MCP server updates automatically
Multiple Tools	Each tool = more code	One MCPToolset, all tools included
API Knowledge Required	Must understand each API	Just connect to server

Benefits of MCP

1. **Separation of Concerns**
    - Agent developers focus on agent logic
    - Tool developers focus on API integration

2. **Reusability**
    - One MCP server can serve multiple agents
    - Tools are standardized across different AI platforms
3. **Maintainability**
    - API changes don't require agent code updates
    - Centralized tool logic
4. **Scalability**
    - Easy to add new tool sources
    - Multiple MCP servers can be connected
5. **Standardization**
    - Consistent tool interface across different services
    - Universal protocol for tool communication

When to Use Each Approach

Use Traditional Function Tools When	Use MCP When
Building quick prototypes	Building production systems
Need custom, complex logic	Standard tools exist
No MCP server available	MCP server is available
Full control required	Want decoupled architecture
Simple, single-tool agents	Multi-tool, complex agents

Running the Agents

Prerequisites

1. **Install Dependencies**

```
pip install -r requirements.txt
```

2. **Set Environment Variables** Create a `.env` file:

```
MAPS_API_KEY=your_google_maps_api_key
GOOGLE_API_KEY=your_gemini_api_key
```

Running Maps_Agent (Traditional)

```
adk run Maps_Agent
```

Running MCP_Maps_Agent (MCP)

```
adk run MCP_Maps_Agent
```

Using ADK Web Interface

```
adk web
```

Then select the agent from the dropdown menu.

Summary

Traditional vs MCP at a Glance

Traditional Approach	**MCP Approach**
Tight coupling between agent and API	**Loose coupling** via standard protocol
~100 lines for 2 tools	~30 lines for 3+ tools
Complexity grows with each tool	Complexity stays constant
Must understand each API	Just connect to server
Full control, full burden	Standard interface, external handling
Good for custom/unique tools	Great for standard services

The Key Insight

```
Traditional: Agent Code ──────────────────────────────► API
             (HTTP logic, parsing, errors, etc.)

MCP:         Agent Code ──► MCP Server ──────────► API
             (just config)   (handles everything)
```

With **2 tools**, our traditional agent already has:

- 100+ lines of code
- 2 different API endpoints
- 4+ error handling points
- Complex JSON parsing logic

The **MCP agent** with **3 tools** has:

- 30 lines of code
- 1 connection configuration
- 0 API-specific logic

As tools grow, the difference becomes dramatic.

MCP represents the future of AI tool integration—a standardized way to connect AI agents with external services, similar to how HTTP standardized web communication or how USB standardized device connections.

Documentation created for educational purposes - teaching Model Context Protocol concepts